

Manual de Aprendizaje: Elixir

Fundamentos, Concurrencia y Proyectos Prácticos

Índice general

1. Preparación del Entorno de Desarrollo e Instalación	3
1.1. Para entornos Windows	3
1.2. Para entornos macOS	3
1.3. Para entornos Linux (Basados en Debian/Ubuntu)	4
1.4. Verificación de la instalación	4
1.5. El entorno interactivo: IEx	4
2. Fundamentos del Lenguaje y Tipos de Datos	5
2.1. Tipos de Datos Básicos	5
2.2. Estructuras de Colección	6
2.3. El Operador de Coincidencia de Patrones (Pattern Matching)	6
3. Módulos, Funciones y Flujo de Datos	7
3.1. Definición de Módulos y Funciones	7
3.2. La Aridad de las Funciones	7
3.3. Coincidencia de Patrones en las Funciones	7
3.4. El Operador de Tubería (Pipe Operator)	8
4. Estructuras de Control de Flujo	9
4.1. 1. Evaluaciones condicionales básicas: if y unless	9
4.2. 2. Condiciones múltiples: cond	9
4.3. 3. Coincidencia de patrones múltiple: case	10
5. Estructuras de Datos Avanzadas y Módulos	11
5.1. Mapas (Maps)	11
5.2. Estructuras (Structs)	11
6. Iteración y Transformación de Colecciones (El Módulo Enum)	13
6.1. Funciones Anónimas	13
6.2. Operaciones principales con Enum	13
6.2.1. 1. Enum.map/2	13
6.2.2. 2. Enum.filter/2	14
6.2.3. 3. Enum.reduce/3	14
7. Creación y Gestión de Proyectos con Mix	15
7.1. Creación de un proyecto nuevo	15
7.2. Compilación y Ejecución	15

8. Introducción a la Concurrency y el Modelo de Actores	17
8.1. Creación de Procesos: spawn	17
8.2. Comunicación entre Procesos: Paso de Mensajes	17
9. Proyecto Práctico - Gestor de Tareas en Consola	19
9.1. 1. Estructura del Proyecto e Inicialización	19
9.2. 2. Modelado de Datos (lib/tarea.ex)	19
9.3. 3. El Motor del Sistema y el Proceso de Estado (lib/motor.ex)	19
9.4. 4. Ejecución e Interacción en el Entorno IEx	21
10. Banco de Ejercicios del Manual	22
10.1. Ejercicios Cortos por Capítulo	22
10.1.1. Capítulo 2: Fundamentos del Lenguaje	22
10.1.2. Capítulo 3 y 4: Funciones y Control de Flujo	22
10.1.3. Capítulo 5 y 6: Estructuras Avanzadas y Colecciones	22
10.1.4. Capítulo 8: Concurrency Básica	22
10.2. Solucionario	23

Capítulo 1

Preparación del Entorno de Desarrollo e Instalación

Antes de iniciar el desarrollo en Elixir, es necesario configurar el entorno de trabajo. Elixir es un lenguaje de programación funcional y concurrente diseñado para ejecutarse sobre la Máquina Virtual de Erlang (conocida como BEAM). Por consiguiente, un requisito estricto para su funcionamiento es contar con Erlang instalado en el sistema. Los métodos de instalación modernos gestionan esta dependencia de forma automatizada.

A continuación, se detallan los procedimientos de instalación para los sistemas operativos más comunes:

1.1. Para entornos Windows

El método estándar en sistemas Windows requiere el uso del instalador oficial provisto por la comunidad de Elixir.

- Navegue al repositorio oficial de lanzamientos de Elixir.
- Descargue el instalador ejecutable más reciente (`elixir-websetup.exe`).
- Ejecute el archivo descargado y siga las instrucciones del asistente de instalación. La configuración predeterminada incluye la instalación automática de Erlang.

1.2. Para entornos macOS

En sistemas macOS, se recomienda utilizar Homebrew, el gestor de paquetes estándar para el entorno de desarrollo de Apple. Desde el emulador de terminal, ejecute el siguiente comando:

```
1 brew install elixir
```

Homebrew resolverá las dependencias correspondientes y procederá a instalar tanto Erlang como Elixir en el sistema.

1.3. Para entornos Linux (Basados en Debian/Ubuntu)

En distribuciones basadas en la arquitectura de Debian, como Ubuntu, la instalación se realiza de forma nativa a través del gestor de paquetes de herramientas avanzadas (`apt`). Desde la terminal, ejecute la siguiente secuencia de comandos:

Actualice los índices de los repositorios del sistema:

```
1 sudo apt update
```

Instale los paquetes de Erlang y Elixir de forma conjunta:

```
1 sudo apt install erlang elixir
```

1.4. Verificación de la instalación

Independientemente del sistema operativo utilizado, es mandatorio comprobar la integridad de la instalación antes de proceder. En la interfaz de línea de comandos (Terminal o Símbolo del Sistema), ejecute:

```
1 elixir -v
```

Una instalación exitosa devolverá en la salida estándar la versión de la máquina virtual Erlang/OTP instalada, seguida de la versión actual del compilador de Elixir.

1.5. El entorno interactivo: IEx

Elixir provee una herramienta de evaluación interactiva denominada IEx (Interactive Elixir), que funciona como un bucle de lectura-evaluación-impresión (REPL). Esta herramienta es fundamental durante la fase de aprendizaje y depuración de código, ya que permite ejecutar expresiones de Elixir en tiempo real sin necesidad de compilar scripts.

Para iniciar la sesión interactiva, ingrese el siguiente comando en la terminal:

```
1 iex
```

El prompt de la terminal cambiará, indicando que el entorno está listo para recibir instrucciones de Elixir (por ejemplo, operaciones aritméticas básicas). Para finalizar la ejecución del proceso IEx y retornar al intérprete de comandos del sistema operativo, es necesario presionar la combinación de teclas `Ctrl+C` dos veces consecutivas.

Capítulo 2

Fundamentos del Lenguaje y Tipos de Datos

Con la herramienta IEx en funcionamiento, es posible comenzar a explorar la sintaxis de Elixir. A diferencia de los lenguajes orientados a objetos, Elixir es un lenguaje funcional donde los datos son inmutables.

A continuación, se describen los tipos de datos primarios y las estructuras de colección fundamentales.

2.1. Tipos de Datos Básicos

Elixir maneja tipos de datos convencionales, junto con algunos específicos del ecosistema de Erlang:

1. **Enteros y Flotantes (Integers and Floats):** Los valores numéricos se expresan de forma estándar. Los números de punto flotante requieren al menos un dígito antes y después del punto decimal.

```
1 x = 10
2 y = 3.14
```

2. **Booleanos (Booleans):** Representan los valores de verdad lógicos.

```
1 true
2 false
```

3. **Átomos (Atoms):** Un átomo es una constante cuyo nombre es su propio valor. Son equivalentes a los símbolos en lenguajes como Ruby. En Elixir, se utilizan extensamente para señalar el estado de una operación (por ejemplo, éxito o error). Se identifican por comenzar con dos puntos (:).

```
1 :ok
2 :error
```

4. **Cadenas de Texto (Strings):** Las cadenas se delimitan con comillas dobles y están codificadas internamente en UTF-8, lo que permite el uso de caracteres especiales de forma nativa.

```
1 "Hola Mundo"
```

2.2. Estructuras de Colección

Para agrupar múltiples valores, Elixir proporciona dos estructuras principales que operan de manera diferente en la memoria:

1. **Listas (Lists):** Se definen utilizando corchetes. Internamente, son listas enlazadas (linked lists). Esto significa que agregar elementos al principio de la lista es una operación muy rápida, pero acceder a un elemento por su índice es una operación lineal.

```
1 [1, 2, 3]
```

2. **Tuplas (Tuples):** Se definen utilizando llaves. Las tuplas se almacenan de forma contigua en memoria. Esto hace que acceder a su tamaño o a un elemento por su índice sea extremadamente rápido, pero modificarlas es costoso porque requiere copiar la tupla entera. Su uso más común es para retornar múltiples valores desde una función.

```
1 {1, 2, 3}
```

2.3. El Operador de Coincidencia de Patrones (Pattern Matching)

En la mayoría de los lenguajes de programación, el signo de igual (=) es un operador de asignación. En Elixir, el signo de igual se denomina operador de coincidencia (*match operator*).

Esta es una diferencia paradigmática crucial. Cuando se utiliza =, Elixir intenta evaluar si el lado izquierdo coincide estructuralmente con el lado derecho.

Ejemplo de asignación inicial:

```
1 x = 1
```

En este caso, Elixir enlaza el valor 1 a la variable x. Hasta aquí, parece una asignación tradicional.

Ejemplo de coincidencia de patrones:

```
1 1 = x
```

Si se ejecuta esta instrucción en IEx, el sistema devolverá 1. No se está asignando x a 1, sino que Elixir verifica si el valor contenido en x coincide con el valor 1. Como la afirmación es verdadera, la coincidencia es exitosa.

Ejemplo de error por no coincidencia:

```
1 2 = x
```

Al ejecutar esta línea, Elixir generará un error (**MatchError**). El sistema indica que el patrón en el lado izquierdo (un 2 literal) no coincide con el valor de la derecha (el valor 1 contenido en x).

Este mecanismo de coincidencia de patrones se utilizará más adelante como la herramienta principal para controlar el flujo del programa y extraer datos de estructuras complejas.

Capítulo 3

Módulos, Funciones y Flujo de Datos

En un lenguaje funcional puro o impuro, la unidad básica de ejecución no es el objeto, sino la función. Elixir no posee clases ni métodos; en su lugar, agrupa funciones relacionadas dentro de constructos denominados módulos.

3.1. Definición de Módulos y Funciones

Para definir un módulo se utiliza la macro `defmodule`, y para definir funciones públicas dentro de ese módulo se utiliza la macro `def`. Si se requiere una función auxiliar que solo pueda ser llamada desde dentro del mismo módulo, se utiliza `defp` (función privada).

Ejemplo de sintaxis básica:

```
1 defmodule Matematicas do
2   def sumar(a, b) do
3     a + b
4   end
5 end
```

Una característica fundamental en Elixir es el retorno implícito. No existe la palabra reservada `return`; una función siempre devolverá el resultado de la última expresión evaluada en su bloque de código. En el ejemplo anterior, la función `sumar` retorna automáticamente el resultado de `a + b`.

3.2. La Aridad de las Funciones

En la terminología de Elixir y Erlang, las funciones se identifican por su módulo, su nombre y su aridad (el número de argumentos que reciben). Esto se anota con el formato `NombreFuncion/Aridad`.

En el ejemplo anterior, la función se denomina formalmente `Matematicas.sumar/2`. Esto es crucial porque Elixir permite definir múltiples funciones con el mismo nombre pero diferente aridad, y el compilador las tratará como funciones completamente distintas.

3.3. Coincidencia de Patrones en las Funciones

El concepto de *Pattern Matching* explorado en el capítulo anterior brilla especialmente en la definición de funciones. Es posible definir la misma función múltiples veces (con

la misma aridad) para manejar diferentes escenarios, dependiendo de los argumentos recibidos.

Elixir evaluará las cláusulas de la función de arriba hacia abajo y ejecutará la primera donde los argumentos coincidan con el patrón.

```
1 defmodule Saludo do
2   # Clausula 1: Coincidencia exacta con un valor especifico
3   def decir_hola("Admin") do
4     "Bienvenido al sistema, Administrador."
5   end
6
7   # Clausula 2: Coincidencia general (actua como un caso por
   defecto)
8   def decir_hola(nombre) do
9     "Hola, " <> nombre # El operador <> concatena cadenas de texto
10  end
11 end
```

Si se llama a `Saludo.decir_hola("Admin")`, se ejecutará la primera cláusula. Si se llama a `Saludo.decir_hola("Carlos")`, la primera cláusula fallará (porque "Carlos" no coincide con "Admin"), pasando a la segunda cláusula donde la variable `nombre` tomará el valor "Carlos".

3.4. El Operador de Tubería (Pipe Operator)

Una de las características más representativas y apreciadas en Elixir es el operador de tubería, representado por `|>`. Este operador resuelve el problema de la lectura anidada de funciones.

Matemáticamente, si se requiere pasar el resultado de una función a otra, la sintaxis tradicional obliga a leer de adentro hacia afuera:

```
1 String.upcase(String.reverse("hola"))
```

El operador `|>` toma el resultado de la expresión a su izquierda y lo inyecta como el primer argumento de la función a su derecha. Esto permite leer las transformaciones de datos como un flujo secuencial, de arriba hacia abajo o de izquierda a derecha:

```
1 "hola"
2 |> String.reverse()
3 |> String.upcase()
```

Esta estructura refleja fielmente el paradigma funcional: los datos ingresan, fluyen a través de una serie de transformaciones puras, y producen un resultado final sin alterar los datos originales.

Capítulo 4

Estructuras de Control de Flujo

A diferencia de los lenguajes de programación imperativos, donde las estructuras de control dirigen el flujo de ejecución sin necesariamente generar un resultado, en Elixir todas las estructuras de control son expresiones; es decir, siempre retornan un valor.

A continuación, se describen los tres mecanismos principales para el control de flujo condicional.

4.1. 1. Evaluaciones condicionales básicas: `if` y `unless`

La macro `if` se utiliza para evaluar condiciones booleanas simples. Si la condición provista es evaluada como verdadera (`true` o cualquier valor distinto de `nil` y `false`), se ejecuta el bloque principal. Opcionalmente, se puede incluir un bloque `else`.

```
1 edad = 18
2
3 resultado = if edad >= 18 do
4   "El usuario es mayor de edad."
5 else
6   "El usuario es menor de edad."
7 end
```

Elixir también provee la macro `unless`, que funciona como el inverso lógico de `if`. El bloque de código dentro de un `unless` solo se ejecuta si la condición evaluada es falsa. Esto resulta útil para mejorar la legibilidad cuando se desean expresar condiciones negativas de forma natural.

```
1 es_administrador = false
2
3 unless es_administrador do
4   "Acceso denegado al panel de control."
5 end
```

4.2. 2. Condiciones múltiples: `cond`

Cuando se requiere evaluar múltiples condiciones independientes (el equivalente a una cadena de sentencias `if`, `else if`, `else` en otros lenguajes), Elixir provee la macro `cond`.

La estructura `cond` evalúa secuencialmente una lista de expresiones y ejecuta el bloque de código correspondiente a la primera condición que resulte verdadera.

```
1 temperatura = 35
2
3 cond do
4   temperatura < 0 ->
5     "Estado: Congelacion"
6   temperatura >= 0 and temperatura < 30 ->
7     "Estado: Normal"
8   temperatura >= 30 ->
9     "Estado: Alerta por calor"
10  true ->
11    "Estado: Desconocido" # Clausula por defecto de seguridad
12 end
```

Nota: Es una buena práctica incluir una cláusula final evaluada como `true` para interceptar cualquier caso no previsto y evitar un error en tiempo de ejecución (`CondClauseError`).

4.3. 3. Coincidencia de patrones múltiple: case

Mientras que `cond` evalúa condiciones booleanas, la macro `case` está diseñada para comparar un único valor contra múltiples patrones utilizando el operador de coincidencia. Es la estructura de control más robusta y utilizada en el ecosistema de Elixir.

El uso clásico de `case` es el manejo de respuestas de funciones que retornan tuplas, lo cual es el estándar para el manejo de errores en Erlang y Elixir.

```
1 # Supongamos una funcion hipotetica que busca un archivo
2 respuesta = {:ok, "Contenido del archivo"}
3
4 case respuesta do
5   {:ok, contenido} ->
6     "Lectura exitosa. El texto es: " <> contenido
7   {:error, razon} ->
8     "Fallo en la lectura. Motivo: " <> razon
9   _ ->
10    "Error desconocido. Patron no coincidente."
11 end
```

En este ejemplo:

1. Elixir intenta emparejar la variable `respuesta` con la primera cláusula `{:ok, contenido}`. Como la estructura coincide (es una tupla de dos elementos que inicia con el átomo `:ok`), la coincidencia es exitosa y la variable local `contenido` adopta el valor de la cadena de texto.
2. Si la respuesta hubiera sido un error, habría saltado a la segunda cláusula.
3. El guion bajo (`_`) actúa como una variable comodín que coincide con cualquier valor, funcionando como un caso por defecto en caso de que los patrones anteriores no se cumplan.

Capítulo 5

Estructuras de Datos Avanzadas y Módulos

Con el conocimiento de listas y tuplas (Capítulo 2), es necesario introducir estructuras capaces de modelar entidades de la vida real mediante el uso de pares clave-valor.

5.1. Mapas (Maps)

El mapa es la estructura de datos asociativa por excelencia en Elixir. Se define utilizando la sintaxis `%{}`, permitiendo utilizar cualquier tipo de dato como clave, aunque el uso de átomos es el estándar para representar atributos conocidos.

```
1 # Definición de un mapa representando un usuario
2 usuario = %{nombre: "Ana", edad: 25, activo: true}
3
4 # Acceso a los valores (cuando las claves son átomos)
5 nombre_usuario = usuario.nombre
6
7 # Actualización de un mapa (Retorna un mapa nuevo, el original es
  immutable)
8 usuario_actualizado = %{usuario | edad: 26}
```

Es imperativo recordar la naturaleza inmutable del lenguaje: la operación de actualización no modifica el mapa `usuario` original en memoria, sino que genera y retorna un nuevo mapa con la clave `edad` modificada.

5.2. Estructuras (Structs)

Mientras que un mapa puede contener cualquier clave de forma arbitraria, una Estructura (Struct) es una extensión de los mapas que provee validación en tiempo de compilación y garantías sobre los campos permitidos.

Las estructuras están siempre vinculadas a un módulo específico que define su `esquema`.

```
1 defmodule Perfil do
2   # Se definen las claves permitidas y sus valores por defecto
3   defstruct nombre: "Anonimo", rol: :usuario
4 end
```

Para instanciar y utilizar esta estructura en el código, se invoca el nombre del módulo junto a la sintaxis de mapa:

```
1 # Creacion utilizando los valores por defecto
2 perfil_base = %Perfil{}
3
4 # Creacion asignando valores especificos
5 perfil_admin = %Perfil{nombre: "Carlos", rol: :administrador}
6
7 # Intentar asignar una clave no definida en 'defstruct' generara
  un error de compilacion
8 # %Perfil{nombre: "Ana", apellido: "Perez"} # Esto fallara
```

Las estructuras son fundamentales para el diseño de software en Elixir, ya que garantizan la consistencia de los datos que fluyen a través de las funciones del sistema.

Capítulo 6

Iteración y Transformación de Colecciones (El Módulo Enum)

En los lenguajes de programación con paradigma imperativo, la iteración se realiza típicamente mediante estructuras de control de bucle, como `for` o `while`, las cuales dependen de la mutación de una variable de estado (por ejemplo, un contador `i++`).

Debido a que Elixir impone la inmutabilidad de los datos, estas estructuras tradicionales no existen. En su lugar, la iteración se resuelve matemáticamente mediante la recursividad y, a un nivel más práctico, utilizando funciones de orden superior provistas por el módulo `Enum`.

El módulo `Enum` contiene una extensa API de algoritmos para procesar cualquier estructura de datos que implemente el protocolo `Enumerable` (como las listas y los mapas).

6.1. Funciones Anónimas

Antes de utilizar `Enum`, es necesario comprender las funciones anónimas, ya que son el argumento principal de las funciones de iteración. Se definen utilizando la sintaxis `fn ->... end`.

```
1 # Definición de una función anónima asignada a una variable
2 multiplicar = fn a, b -> a * b end
3
4 # Invocación de la función anónima (requiere el uso de un punto
   previo a los parentesis)
5 resultado = multiplicar.(5, 3)
```

6.2. Operaciones principales con Enum

Las operaciones más comunes en el procesamiento de datos son la proyección (mapeo), la restricción (filtrado) y la agregación (reducción).

6.2.1. 1. Enum.map/2

Aplica una función dada a cada elemento de la colección y retorna una nueva colección con los resultados transformados, manteniendo la misma longitud que la original.

```

1 numeros = [1, 2, 3, 4]
2 cuadrados = Enum.map(numeros, fn x -> x * x end)
3 # Retorna: [1, 4, 9, 16]

```

6.2.2. 2. Enum.filter/2

Evalúa cada elemento de la colección mediante una función predicado (una función que retorna un valor booleano). Retorna una nueva colección que contiene únicamente los elementos para los cuales la función evaluó como verdadero.

```

1 numeros = [1, 2, 3, 4, 5, 6]
2 pares = Enum.filter(numeros, fn x -> rem(x, 2) == 0 end)
3 # Retorna: [2, 4, 6]

```

6.2.3. 3. Enum.reduce/3

Itera sobre la colección acumulando un valor final. Recibe la colección, un valor inicial (acumulador) y una función que define cómo el elemento actual modificará dicho acumulador.

```

1 numeros = [1, 2, 3, 4]
2 valor_inicial = 0
3 suma_total = Enum.reduce(numeros, valor_inicial, fn elemento,
4   acumulador ->
5   acumulador + elemento
6   end)
7 # Retorna: 10

```

Estas funciones, combinadas con el operador de tubería (`|>`) examinado en el Capítulo 3, permiten procesar transformaciones de datos complejas de forma declarativa y altamente legible.

Capítulo 7

Creación y Gestión de Proyectos con Mix

Hasta este punto, la ejecución de código se ha realizado exclusivamente a través de IEx (Interactive Elixir) o mediante scripts aislados. Sin embargo, para el desarrollo de software a nivel de producción, es necesario estructurar el código en directorios, gestionar dependencias externas y compilar artefactos.

Para resolver estas necesidades, Elixir incluye su propia herramienta de construcción y gestión de proyectos denominada Mix.

7.1. Creación de un proyecto nuevo

Para inicializar la estructura estándar de un proyecto, se debe utilizar la interfaz de línea de comandos del sistema operativo (fuera de la sesión IEx). Ejecute el siguiente comando, reemplazando `nombre_proyecto` por el identificador deseado:

```
1 mix new nombre_proyecto
```

Este comando generará un nuevo directorio con una jerarquía de archivos estandarizada. La estructura principal resultante es la siguiente:

- `/lib`: Es el directorio principal del código fuente. Aquí residirán todos los módulos (`.ex`) de la aplicación.
- `/test`: Contiene los archivos de pruebas unitarias (`.exs`). Elixir fomenta el desarrollo guiado por pruebas (TDD) y proporciona la herramienta ExUnit nativamente.
- `mix.exs`: Es el archivo de configuración central del proyecto. Contiene metadatos de la aplicación, la versión requerida de Elixir y la lista de dependencias de terceros (equivalente a un `package.json` en Node.js o `pom.xml` en Java).

7.2. Compilación y Ejecución

Una vez situado dentro del directorio del proyecto recién creado (`cd nombre_proyecto`), es posible compilar el código fuente ejecutando:

```
1 mix compile
```

Para interactuar con los módulos definidos en el directorio `/lib` y probar su comportamiento en tiempo real, se debe iniciar la consola interactiva indicándole que cargue el contexto del proyecto actual. Esto se logra con el modificador `-S`:

```
1 iex -S mix
```

Al ejecutar este comando, el entorno IEx compilará automáticamente cualquier cambio reciente en los archivos y permitirá invocar las funciones del proyecto de manera inmediata, facilitando un ciclo de desarrollo y depuración altamente eficiente.

Capítulo 8

Introducción a la Concurrencia y el Modelo de Actores

La razón principal por la que sistemas de alta disponibilidad adoptan Elixir es su capacidad para manejar miles o millones de tareas simultáneas sin agotar los recursos del sistema operativo. Esto se logra mediante la implementación del Modelo de Actores.

En Elixir, todo el código se ejecuta dentro de procesos. Es imperativo aclarar que estos no son procesos del sistema operativo ni hilos (threads) tradicionales; son procesos ligeros gestionados internamente por la máquina virtual BEAM. Están aislados entre sí, no comparten memoria y son extremadamente económicos en términos de consumo de recursos.

8.1. Creación de Procesos: `spawn`

Para iniciar un nuevo proceso concurrente, se utiliza la función `spawn`. Esta función recibe como argumento una función anónima, la ejecuta en un proceso independiente y retorna un Identificador de Proceso (PID).

```
1 # Se crea un proceso que evalua una expresion y termina su
   ejecucion inmediatamente
2 pid = spawn(fn -> IO.puts("Ejecucion desde un proceso ligero
   aislado") end)
3
4 # El sistema retorna el identificador unico del proceso, por
   ejemplo:
5 # #PID<0.109.0>
```

8.2. Comunicación entre Procesos: Paso de Mensajes

Dado que los procesos en Elixir no comparten memoria, la única forma en que pueden interactuar o compartir estado es a través del paso de mensajes (*message passing*).

Cada proceso posee un buzón de entrada (*mailbox*). Para enviar un mensaje al buzón de un proceso, se utiliza la macro `send/2`, la cual requiere el PID del proceso destino y el mensaje (usualmente estructurado como una tupla).

```
1 # Enviando un mensaje al proceso actual (self) para propositos de
   demostracion
```

```
2 send(self(), {:notificacion, "El sistema ha iniciado correctamente"
  })
```

Para procesar los mensajes almacenados en el buzón, el proceso receptor debe utilizar la estructura `receive`. Esta estructura utiliza internamente el mecanismo de coincidencia de patrones (*Pattern Matching*) para identificar y extraer la información del mensaje entrante.

```
1 receive do
2   {:notificacion, mensaje} ->
3     IO.puts("Mensaje recibido: " <> mensaje)
4   {:error, motivo} ->
5     IO.puts("Se ha recibido una alerta de error.")
6 after
7   5000 ->
8     # Clausula de seguridad para evitar bloqueos infinitos (
9     # Timeout en milisegundos)
10    IO.puts("Tiempo de espera agotado. No se recibieron mensajes."
11    )
12 end
```

Nota de Arquitectura: Aislamiento vs. Tolerancia a Fallas: Es un error común asumir que el uso exclusivo de `spawn` dota a la aplicación de tolerancia a fallos automática.

El aislamiento garantiza que si un proceso muere por un error no controlado, no arrastrará consigo al resto del sistema. Sin embargo, la verdadera resiliencia en Elixir se logra mediante OTP (Open Telecom Platform) y el uso de:

- **Procesos enlazados (Links):** Mecanismos donde la caída de un proceso notifica a otro.
- **Monitores:** Permiten observar el ciclo de vida de un proceso de forma unidireccional.
- **Supervisores:** Procesos especializados cuyo único propósito es gestionar el ciclo de vida de otros procesos (hijos).

Si un proceso decae debido a una excepción, un supervisor intercepta el fallo y, aplicando una estrategia de reinicio predefinida, vuelve a instanciar el proceso en un estado limpio. Esta organización es el pilar de la filosofía "Let it crash" (Déjalo colapsar): en lugar de defender el código defensivamente con infinitos bloques `try/catch`, se permite que el proceso falle de forma aislada, confiando en que el supervisor restaura la normalidad de inmediato.

Capítulo 9

Proyecto Práctico - Gestor de Tareas en Consola

Para consolidar los conocimientos adquiridos a lo largo de este manual, construiremos una aplicación funcional de gestión de tareas que se ejecuta en la terminal utilizando los procesos del sistema para mantener el estado de los datos.

9.1. 1. Estructura del Proyecto e Inicialización

Primero, genere el proyecto utilizando Mix desde su terminal:

```
1 mix new gestor_tareas
```

Dentro de la carpeta `lib/`, crearemos un archivo llamado `tarea.ex` para modelar nuestra estructura de datos mediante un struct.

9.2. 2. Modelado de Datos (`lib/tarea.ex`)

```
1 defmodule GestorTareas.Tarea do
2   @moduledoc """
3   Define la estructura de una tarea dentro del sistema.
4   """
5   defstruct [:id, :titulo, completada: false]
6 end
```

9.3. 3. El Motor del Sistema y el Proceso de Estado (`lib/motor.ex`)

Dado que los datos en Elixir son inmutables, utilizaremos un proceso interactivo basado en recursividad para almacenar la lista de tareas en memoria.

```
1 defmodule GestorTareas.Motor do
2   alias GestorTareas.Tarea
3
4   @doc """
5   Inicia el proceso del motor con una lista vacia de tareas.
```

```

6  """
7  def iniciar do
8      spawn(fn -> bucle([]) end)
9  end
10
11  # Bucle recursivo que mantiene el estado de la lista
12  defp bucle(tareas) do
13      receive do
14          {:agregar, titulo, de_quien} ->
15              nueva_id = Enum.count(tareas) + 1
16              nueva_tarea = %Tarea{id: nueva_id, titulo: titulo}
17              nuevas_tareas = [nueva_tarea | tareas] # Agregado
18              eficiente al inicio
19              send(de_quien, {:ok, "Tarea agregada con ID: #{nueva_id}"})
20          })
21          bucle(nuevas_tareas)
22
23          {:completar, id, de_quien} ->
24              case buscar_y_completar(tareas, id) do
25                  {:ok, lista_actualizada} ->
26                      send(de_quien, {:ok, "Tarea #{id} marcada como
27                          completada."})
28                      bucle(lista_actualizada)
29                      {:error, mensaje} ->
30                          send(de_quien, {:error, mensaje})
31                          bucle(tareas)
32              end
33
34          {:listar, de_quien} ->
35              send(de_quien, {:lista, tareas})
36              bucle(tareas)
37      end
38  end
39
40  # Funciones auxiliares usando transformacion de colecciones y
41  # Pattern Matching
42  defp buscar_y_completar(tareas, id) do
43      if Enum.any?(tareas, fn t -> t.id == id end) do
44          lista_nueva = Enum.map(tareas, fn t ->
45              if t.id == id, do: %{t | completada: true}, else: t
46              end)
47          {:ok, lista_nueva}
48      else
49          {:error, "No se encontro la tarea con ID: #{id}"}
50      end
51  end

```

9.4. 4. Ejecución e Interacción en el Entorno IEx

Para interactuar con el proyecto, cargue el entorno Mix dentro de IEx desde la terminal:

```
1 iex -S mix
```

Ejecute la siguiente secuencia de comandos para probar el flujo completo:

```
1 # 1. Iniciar el motor que guardara el estado
2 pid = GestorTareas.Motor.iniciar()
3
4 # 2. Agregar tareas enviando mensajes al proceso
5 send(pid, {:agregar, "Estudiar Pattern Matching", self()})
6 receive do {:ok, msg} -> IO.puts(msg) end
7
8 send(pid, {:agregar, "Escribir codigo en Elixir", self()})
9 receive do {:ok, msg} -> IO.puts(msg) end
10
11 # 3. Listar tareas vigentes
12 send(pid, {:listar, self()})
13 receive do {:lista, lista} -> IO.inspect(lista) end
14
15 # 4. Completar una tarea especifica
16 send(pid, {:completar, 1, self()})
17 receive do {:ok, msg} -> IO.puts(msg) end
```

Capítulo 10

Banco de Ejercicios del Manual

10.1. Ejercicios Cortos por Capítulo

10.1.1. Capítulo 2: Fundamentos del Lenguaje

1. **Modelado de un Producto:** Cree una tupla que represente un producto de una tienda. Debe contener el nombre del artículo (como un string), su precio (como flotante) y su disponibilidad en inventario (como booleano).
2. **Uso de Átomos:** Defina un mapa simple que represente la respuesta de una solicitud web, utilizando átomos para las claves `estatus` y `codigo`.

10.1.2. Capítulo 3 y 4: Funciones y Control de Flujo

3. **Determinación de Números Pares:** Escriba un módulo llamado `Verificador` con una función `par?/1` que reciba un entero y devuelva `true` si es par y `false` en caso contrario.
4. **Estructuras Condicionales:** Desarrolle una función utilizando `case` que reciba una tupla de estado como `{:ok, datos}` o `{:error, razon}` y regrese un mensaje amigable para el usuario.

10.1.3. Capítulo 5 y 6: Estructuras Avanzadas y Colecciones

5. **Filtrado de Usuarios Activos:** Teniendo una lista de mapas donde cada mapa representa a un usuario con los campos `nombre` y `activo` (booleano), use el módulo `Enum` para extraer únicamente los usuarios activos.
6. **Cálculo de Ventas Totales:** Dada una lista de números que representan las ventas del día `[12.50, 45.00, 9.99, 100.00]`, calcule el total general utilizando `Enum.reduce/3`.

10.1.4. Capítulo 8: Concurrencia Básica

7. **El Proceso Eco:** Desarrolle un proceso síncrono mediante `spawn` y `receive` que escuche permanentemente mensajes. Si recibe un string, debe imprimir en pantalla: "Mensaje recibido de forma segura: [Texto]".

10.2. Solucionario

```
1 # Solucion Ejercicio 1 (Tuplas)
2 producto = {"Teclado Mecanico", 89.99, true}
3
4 # Solucion Ejercicio 2 (Mapas y Atomos)
5 respuesta_web = %{estatus: :exito, codigo: 200}
6
7 # Solucion Ejercicio 3 (Modulos y funciones)
8 defmodule Verificador do
9   def par?(numero) do
10     rem(numero, 2) == 0
11   end
12 end
13
14 # Solucion Ejercicio 4 (Case/Pattern matching)
15 defmodule Evaluador do
16   def revisar_respuesta(resultado) do
17     case resultado do
18       {:ok, datos} -> "Operacion exitosa con los datos: #{datos}"
19       {:error, razon} -> "Error detectado. Motivo del fallo: #{
20         razon}"
21     end
22   end
23 end
24
25 # Solucion Ejercicio 5 (Enum.filter)
26 defmodule AdministradorUsuarios do
27   def obtener_activos(usuarios) do
28     Enum.filter(usuarios, fn u -> u.activo == true end)
29   end
30 end
31
32 # Prueba: AdministradorUsuarios.obtener_activos([%{nombre: "Ana",
33   activo: true}, %{nombre: "Luis", activo: false}])
34
35 # Solucion Ejercicio 6 (Enum.reduce)
36 defmodule Contabilidad do
37   def calcular_total(ventas) do
38     Enum.reduce(ventas, 0.0, fn venta, acumulador -> venta +
39       acumulador end)
40   end
41 end
42
43 # Solucion Ejercicio 7 (Proceso Eco interactivo)
44 defmodule ProcesoEco do
45   def escuchar do
46     receive do
47       mensaje when is_binary(mensaje) ->
48         IO.puts("Mensaje recibido de forma segura: #{mensaje}")
49         escuchar() # Llamada recursiva para mantener el proceso
50     end
51   end
52 end
```

```
46     - ->
47     IO.puts("Tipo de dato no soportado.")
48     escuchar()
49   end
50 end
51 end
52
53 # Para iniciar: pid_eco = spawn(&ProcesoEco.escuchar/0)
54 # Para probar: send(pid_eco, "Hola Elixir")
```

Bibliografía

- [1] D. Thomas, *Programming Elixir ≥ 1.6: Functional />Concurrent />Pragmatic />Fun*. Raleigh, NC, USA: Pragmatic Bookshelf, 2018. [Online]. Available: <https://pragprog.com/titles/elixir16/programming-elixir-1-6/>
- [2] S. Jurić, *Elixir in Action*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2019. [Online]. Available: <https://www.manning.com/books/elixir-in-action-second-edition>
- [3] B. Marx, J. Valim, and B. Tate, *Adopting Elixir: From Concept to Production*. Raleigh, NC, USA: Pragmatic Bookshelf, 2018. [Online]. Available: <https://pragprog.com/titles/tvmelixir/adopting-elixir/>
- [4] C. McCord, B. Tate, and J. Valim, *Programming Phoenix 1.4*. Raleigh, NC, USA: Pragmatic Bookshelf, 2019. [Online]. Available: <https://pragprog.com/titles/phoenix14/programming-phoenix-1-4/>
- [5] Elixir Core Team, “Elixir Official Documentation,” HexDocs, 2024. [Online]. Available: <https://hexdocs.pm/elixir/>